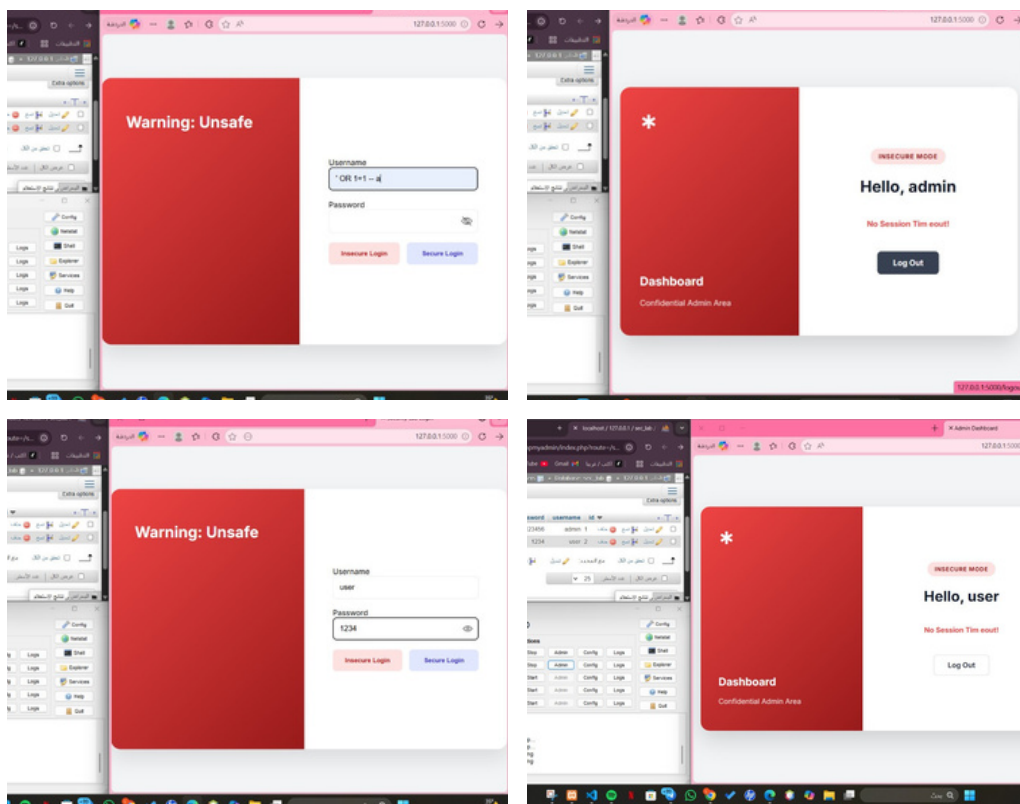


SOFTWARE SECURITY LABS REPORT

Lama Aljohani

LAB 1: Web Application: Insecure Login System

Part 1: Insecure Implementation (Interfaces)



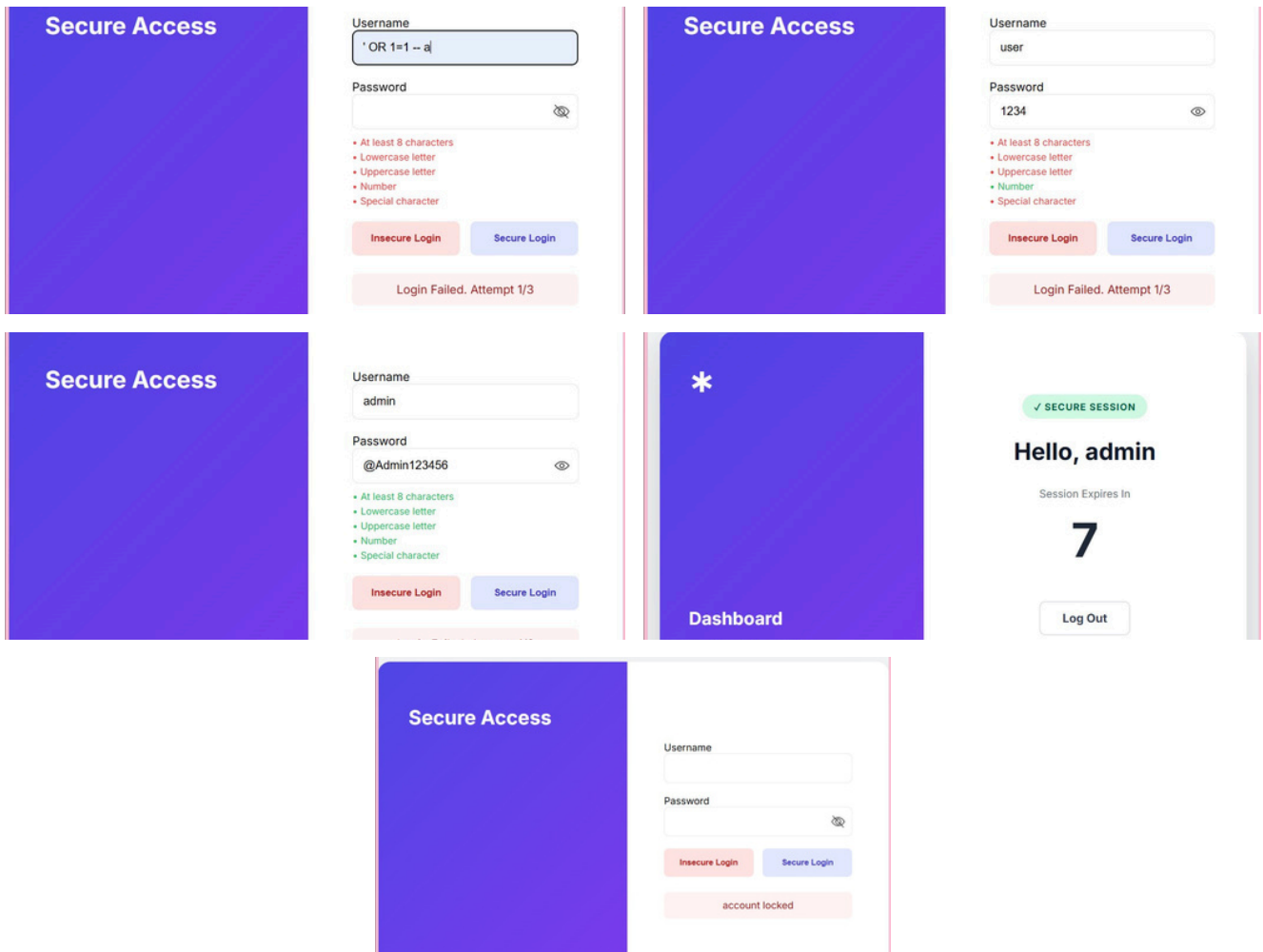
[Image 1: SQL Injection Attempt]

[Image 2: Successful SQLi Login to Admin]

[Image 3: Entering a Weak Password]

[Image 4: Login Success with Weak Password]

I used MySQL to store user data and I tested an SQL injection attack using ' OR 1=1--a and bypassed the login. I also entered a weak password like "1234", and the system accepted it. There is no session management or timeout on the dashboard pages.



Part 2: Secure Implementation (Interfaces)

[Image 5: Blocked SQL Injection]

[Image 6: Password Policy Requirements Error]

[Image 7: Entering a Strong Password]

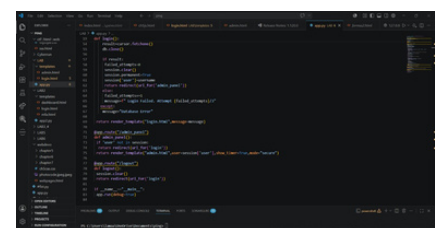
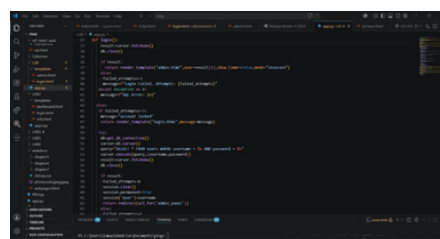
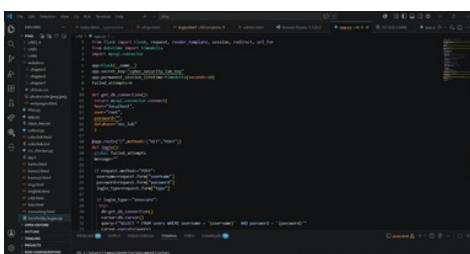
[Image 8: Secure Dashboard with Session Timer]

[Image 9: Account Locked Message]

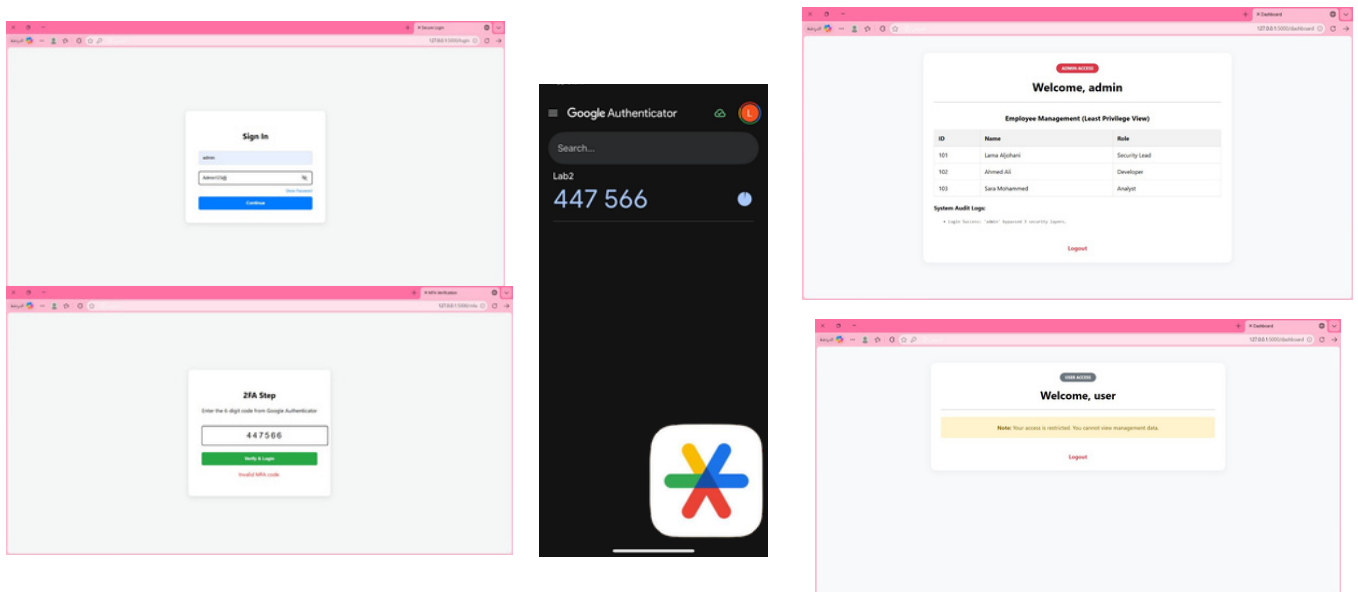
The secure system blocked the SQL injection attack. It rejected the weak password and showed the complexity rules. After a secure login, the dashboard has a 10-second timer. The system also locked the account after 3 failed attempts.

Part 3: Code Implementation

The insecure code accepts direct input into the MySQL query. The secure code uses input validation to stop SQL injection. It also uses RegEx for password rules and a timer for session management.



LAB 2: Core & Design Security Concepts



The system uses a password and a 6-digit code from Google Authenticator for login. I also applied the Principle of Least Privilege, where only the admin can see the employee table.

```
def login():
    username = request.args.get('username')
    password = request.args.get('password')
    if not username or not password:
        return redirect(url_for('login'))
    user = User.query.filter_by(username=username).first()
    if not user:
        return redirect(url_for('login'))
    if not user.check_password(password):
        return redirect(url_for('login'))
    session['user'] = user
    session['role'] = user.role
    if user.role == 'admin':
        return redirect(url_for('dashboard'))
    else:
        return redirect(url_for('restricted_view'))
```

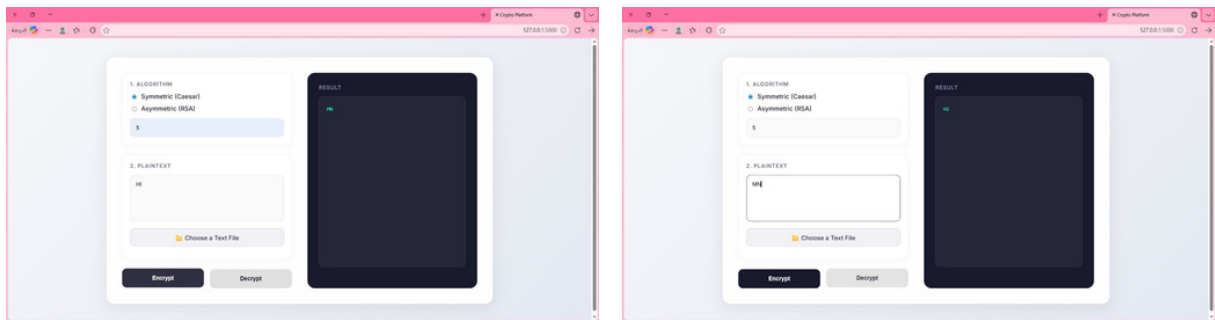
```
def mfa():
    code = request.args.get('code')
    if not code:
        return redirect(url_for('login'))
    user = session.get('user')
    if not user:
        return redirect(url_for('login'))
    if not user.verify_2fa(code):
        return redirect(url_for('login'))
    session['mfa_verified'] = True
    return redirect(url_for('dashboard'))
```

```
def mfa():
    code = request.args.get('code')
    if not code:
        return redirect(url_for('login'))
    user = session.get('user')
    if not user:
        return redirect(url_for('login'))
    if not user.verify_2fa(code):
        return redirect(url_for('login'))
    session['mfa_verified'] = True
    return redirect(url_for('dashboard'))

@app.route('/dashboard')
def dashboard():
    if 'user' not in session:
        return redirect(url_for('login'))
    data = EMPLOYEES if session['role'] == 'admin' else []
    return render_template('dashboard.html',
        user=session['user'],
        role=session['role'],
        employees=data,
        logs=security_audit_log)
```

The code verifies the MFA code and checks the user role. It hides sensitive data if the user is not an admin.

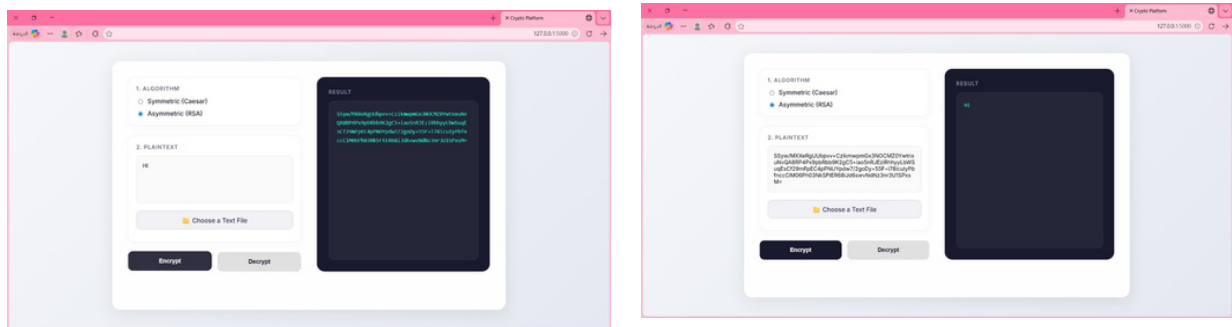
LAB 3: Symmetric Cryptography



Caesar Cipher Interface

I used the Caesar Cipher with key 5 to encrypt "HI" into "MN". The same key is used to decrypt it back.

LAB 4: Asymmetric Cryptography



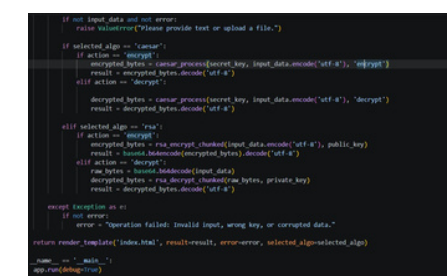
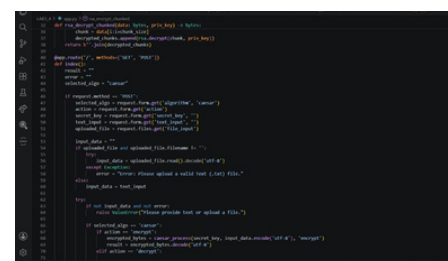
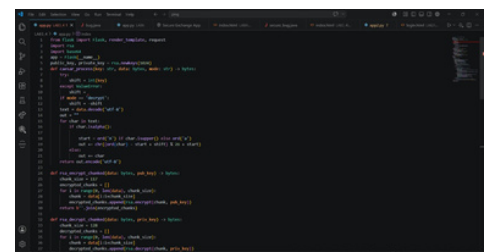
[Insert RSA Interface]

I used RSA for asymmetric encryption. It uses a public key to encrypt and a private key to decrypt. I also verified the calculation manually.

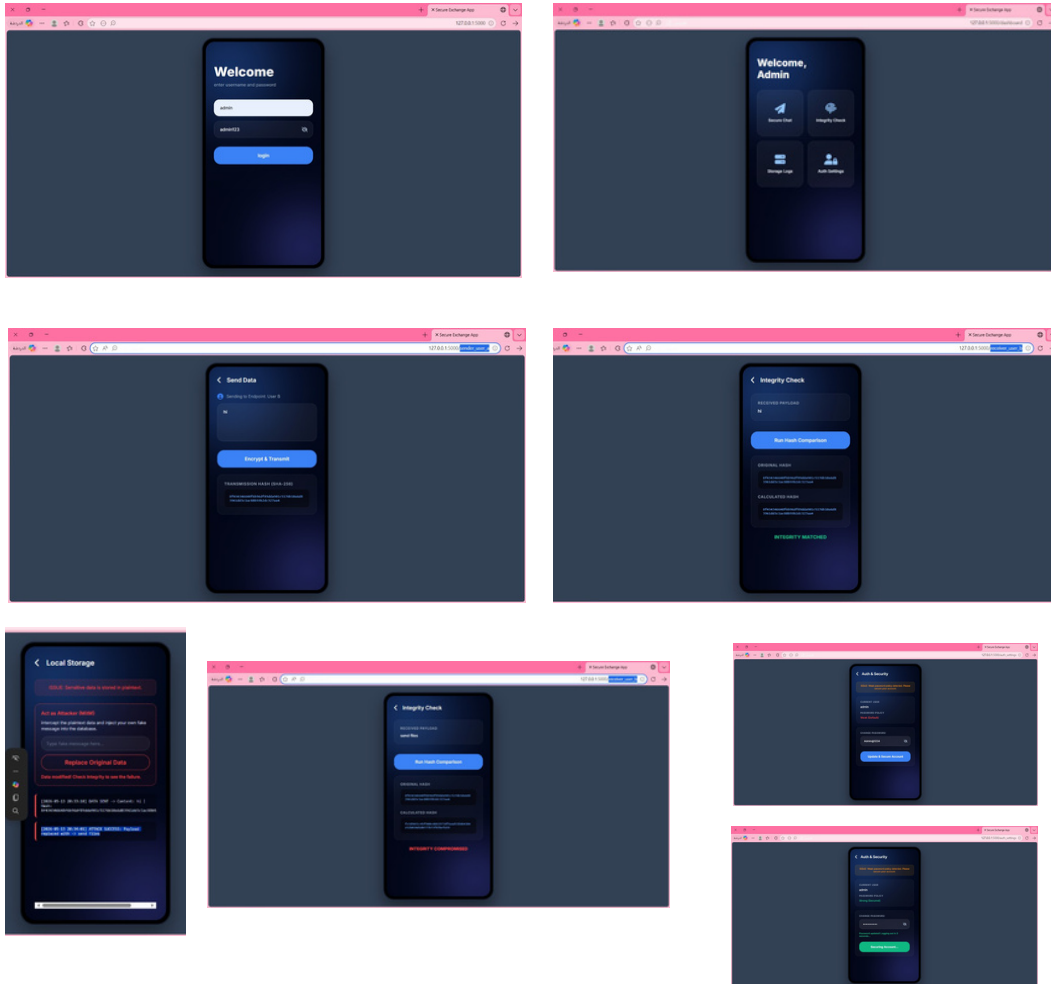
Handwritten RSA calculations:

Caesar:
Key: 5
ABCDEF GHIJ KLMNOPQRSTUVWXYZ
Input: HI
Output: MN

RSA:
1. $p = 17, q = 113$
2. $\phi(n) = (p-1) \times (q-1) = 10 \times 112 = 1120$
3. $e = 7, \text{ gcd}(e, \phi(n)) = 1$
4. $d = e^{-1} \pmod{\phi(n)} = 7^{-1} \pmod{1120} = 161$
Public Key: $(e, n) = (7, 1961)$
Private Key: $(d, n) = (161, 1961)$
4. $C = M^e \pmod{n}$
 $M = 72, 72^7 \pmod{1961} = 19$
 $M = 73, 73^7 \pmod{1961} = 83$
5. $M = C^d \pmod{n}$
 $C = 19, 19^{161} \pmod{1961} = 72$
 $C = 83, 83^{161} \pmod{1961} = 73$



LAB 6: Mobile Application Security Issues



Data in Transit and integrity verification. To demonstrate secure communication, a message ("hi") was successfully transmitted from User A to User B, which is clearly indicated by the URL dynamically changing from the sender's endpoint to the receiver's endpoint. During this exchange, the Python backend generates a secure SHA-256 hash for the payload. Once the message reaches the receiver, a strict integrity check is performed; the system compares the original hash with the calculated hash and verifies that the data arrived completely unaltered. To test this defense mechanism, I simulated a Man-in-the-Middle (MitM) attack by exploiting insecure local storage to inject a modified payload ("send files"). As expected, the receiver's integrity check immediately detected the tampering, failed the hash comparison, and triggered an "INTEGRITY COMPROMISED" alert, proving the application's robust capability to detect unauthorized modifications during data transmission. Additionally, basic authentication security was briefly addressed by enforcing a stronger password policy

```
def login(request):
    username = request.POST.get('username')
    password = request.POST.get('password')

    if not username or not password:
        return HttpResponse('Please enter username and password')

    user = User.objects.filter(username=username).first()
    if not user:
        return HttpResponse('User not found')

    if not check_password(password, user.password):
        return HttpResponse('Password incorrect')

    request.session['username'] = username
    return HttpResponseRedirect(reverse('home'))

def transfer_data(request):
    data = request.POST.get('data')
    if not data:
        return HttpResponse('Data is required')

    # Generate SHA-256 hash
    hash = hashlib.sha256(data.encode()).hexdigest()

    # Store data and hash in session
    request.session['data'] = data
    request.session['hash'] = hash

    return HttpResponseRedirect(reverse('integrity_check'))

def integrity_check(request):
    data = request.session.get('data')
    hash = request.session.get('hash')

    if not data or not hash:
        return HttpResponse('Data or hash is missing')

    # Calculate hash of received data
    received_hash = hashlib.sha256(data.encode()).hexdigest()

    # Compare hashes
    if hash != received_hash:
        return HttpResponse('INTEGRITY COMPROMISED')

    return HttpResponse('Data is intact')
```

```
def login(request):
    username = request.POST.get('username')
    password = request.POST.get('password')

    if not username or not password:
        return HttpResponse('Please enter username and password')

    user = User.objects.filter(username=username).first()
    if not user:
        return HttpResponse('User not found')

    if not check_password(password, user.password):
        return HttpResponse('Password incorrect')

    request.session['username'] = username
    return HttpResponseRedirect(reverse('home'))

def transfer_data(request):
    data = request.POST.get('data')
    if not data:
        return HttpResponse('Data is required')

    # Generate SHA-256 hash
    hash = hashlib.sha256(data.encode()).hexdigest()

    # Store data and hash in session
    request.session['data'] = data
    request.session['hash'] = hash

    return HttpResponseRedirect(reverse('integrity_check'))

def integrity_check(request):
    data = request.session.get('data')
    hash = request.session.get('hash')

    if not data or not hash:
        return HttpResponse('Data or hash is missing')

    # Calculate hash of received data
    received_hash = hashlib.sha256(data.encode()).hexdigest()

    # Compare hashes
    if hash != received_hash:
        return HttpResponse('INTEGRITY COMPROMISED')

    return HttpResponse('Data is intact')
```